

CAPE COLLEGE OF ENGINEERING ALAPPUZHA

Department of _____

PCCSL 507 MACHINE LEARNING LAB

LABORATORY RECORD

Submitted by

Name : _____

Reg. No. : _____

Programme : _____

Semester / Year : _____

Academic Year : _____

Faculty In-Charge : _____

Signature : _____ Date : _____

INDEX

Sl. No.	Title of the Experiment	Date	Page No.	Signature
1A	NumPy Basics – Student Marks Analysis		5	
1B	Pandas DataFrame – Student Details		9	
2	NumPy 2D Array – Branch Sales Analysis		13	
3A	Simple Linear Regression – Study Hours		17	
3B	Multiple Linear Regression – House Price		21	
4A	Simple Linear Regression (Least Squares)		25	
4B	Simple Linear Regression (Gradient Descent)		29	
4C	Simple Linear Regression with Scikit-learn		35	
9				
10				
11				
12				
13				
14				
15				

EXPERIMENT NO. 1A

Date: _____

Page No.: _____

TITLE

NumPy Basics – Student Marks Analysis

OBJECTIVE

To familiarize with NumPy array equations by storing the marks of 3 students in 5 subjects and finding the total, average, highest score and subject-wise average.

AIM

To store the marks of 3 students in 5 subjects in a NumPy array and compute the total, average, highest score and subject-wise average.

THEORY

Marks matrix (3 students $\times$ 5 subjects):

$$M = \begin{bmatrix} m_{11} & \cdots & m_{15} \\ m_{21} & \cdots & m_{25} \\ m_{31} & \cdots & m_{35} \end{bmatrix}$$

Total of all marks:

$$T = \sum_{i=1}^3 \sum_{j=1}^5 m_{ij} \quad (\text{np.sum})$$

Overall average:

$$\bar{A} = \frac{T}{15} \quad (\text{np.mean})$$

Highest score and its position:

$$H = \max(M) \quad (\text{np.max, np.argmax})$$

Student-wise total (row sum, axis=1):

$$T_i = \sum_{j=1}^5 m_{ij}$$

Subject-wise average (column mean, axis=0):

$$\bar{S}_j = \frac{1}{3} \sum_{i=1}^3 m_{ij}$$

PROCEDURE

1. Store the marks of 3 students in 5 subjects in a 3×5 NumPy array.
2. Find the total (np.sum) and the average (np.mean) of all marks.
3. Find the highest score (np.max) and its student/subject (np.argmax).
4. Find the student-wise total (sum, axis=1) and subject-wise average (mean, axis=0).
5. Display all results and verify them.

DATASET DESCRIPTION

Dataset Name: Student marks (synthetic, entered in program)

Source: 3 students $\times$ 5 subjects, marks out of 100

Number of Samples: 15 marks Number of Features: 5 subjects

Target / Class: Not applicable (descriptive statistics)

Description: S1=[78, 85, 92, 88, 76]; S2=[65, 72, 80, 69, 74]; S3=[90, 88, 95, 91, 89].

PROGRAM

```
import numpy as np

marks = np.array([
    [78, 85, 92, 88, 76],
    [65, 72, 80, 69, 74],
    [90, 88, 95, 91, 89],
])

print("Marks (3 students x 5 subjects):")
print(marks)

total = marks.sum()
average = marks.mean()
highest = marks.max()
pos = np.unravel_index(marks.argmax(), marks.shape)
print("Total =", total)
print("Average =", round(float(average), 2))
print("Highest score =", highest, "by student", pos[0] + 1,
      "in subject", pos[1] + 1)

stud_total = marks.sum(axis=1)
print("Student-wise total =", stud_total)
sub_avg = marks.mean(axis=0)
print("Subject-wise average =", np.round(sub_avg, 2))
```

OUTPUT

Marks (3 students x 5 subjects):

```
[[78 85 92 88 76]
 [65 72 80 69 74]
 [90 88 95 91 89]]
```

Total = 1232

Average = 82.13

Highest score = 95 by student 3 in subject 3

Student-wise total = [419 360 453]

Subject-wise average = [77.67 81.67 89. 82.67 79.67]

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **NumPy array statistics** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 1B

Date: _____

Page No.: _____

TITLE

Pandas DataFrame – Student Details

OBJECTIVE

To work with a Pandas DataFrame of five students (roll no, name, department, marks) and perform selection, filtering, aggregation and CSV export operations.

AIM

To create a student dataframe, display rows/columns, filter rows, compute summary statistics with department-wise average, and save it as CSV.

THEORY

DataFrame with rows and named columns:

$$\text{df} = \text{pd.DataFrame}(\{\text{columns}\}) \quad \text{df.head}(n), \text{df.tail}(n)$$

Column selection and boolean filtering:

$$\text{df}[["\text{Name}", "\text{Marks}"]] \quad \text{df}[\text{df}["\text{Marks}] > 80]$$

Summary statistics:

$$\text{max}, \text{min}, \text{mean} = \frac{1}{n} \sum x_i, \text{sum} = \sum x_i$$

Department-wise average and CSV export:

$$\text{df.groupby}(\text{"Department"})["\text{Marks}"].\text{mean}() \quad \text{df.to_csv}(\text{file})$$
PROCEDURE

1. Create a DataFrame of 5 students with Roll no, Name, Department, Marks and display it.
2. Display the first 3 rows (head), last 2 rows (tail) and only Name/Marks columns.
3. Filter students with marks above 80 and students of CSE department.
4. Compute highest, lowest, average and total marks; average mark department-wise.
5. Save the DataFrame to CSV and verify the file.

DATASET DESCRIPTION

Dataset Name: Student details (entered in program)

Source: 5 students; departments CSE, ECE, ME

Number of Samples: 5 rows Number of Features: 4 columns

Target / Class: Not applicable (data handling operations)

Description: Marks range 68–90; exported to exp3_students.csv.

PROGRAM

```
import pandas as pd

data = {
    "Roll no": [1, 2, 3, 4, 5],
    "Name": ["Anu", "Ravi", "Meera", "Arjun", "Divya"],
    "Department": ["CSE", "ECE", "CSE", "ME", "CSE"],
    "Marks": [85, 72, 90, 68, 78],
}

df = pd.DataFrame(data)
print("Dataframe:")
print(df.to_string(index=False))

print("\nFirst 3 rows:")
print(df.head(3).to_string(index=False))
print("\nLast two rows:")
print(df.tail(2).to_string(index=False))
print("\nName and Marks columns:")
print(df[["Name", "Marks"]].to_string(index=False))
print("\nMarks more than 80:")
print(df[df["Marks"] > 80].to_string(index=False))
print("\nCSE department:")
print(df[df["Department"] == "CSE"].to_string(index=False))
print("\nHighest =", df["Marks"].max(), " Lowest =", df["Marks"].min(),
      " Avg =", round(df["Marks"].mean(), 2), " Total =", df["Marks"].sum())
print("\nAverage mark department wise:")
print(df.groupby("Department")["Marks"].mean().round(2).to_string())

df.to_csv("exp3_students.csv", index=False)
print("\nSaved to exp3_students.csv")
```

OUTPUT

Dataframe:

Roll no	Name	Department	Marks
1	Anu	CSE	85
2	Ravi	ECE	72
3	Meera	CSE	90
4	Arjun	ME	68
5	Divya	CSE	78

First 3 rows:

Roll no	Name	Department	Marks
1	Anu	CSE	85
2	Ravi	ECE	72
3	Meera	CSE	90

Last two rows:

Roll no	Name	Department	Marks
4	Arjun	ME	68
5	Divya	CSE	78

Name and Marks columns:

Name	Marks
Anu	85
Ravi	72
Meera	90
Arjun	68
Divya	78

Marks more than 80:

Roll no	Name	Department	Marks
1	Anu	CSE	85
3	Meera	CSE	90

CSE department:

Roll no	Name	Department	Marks
1	Anu	CSE	85
3	Meera	CSE	90
5	Divya	CSE	78

Highest = 90 Lowest = 68 Avg = 78.6 Total = 393

Average mark department wise:

Department	
CSE	84.33
ECE	72.00
ME	68.00

Saved to exp3_students.csv

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **Pandas DataFrame operations** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 2

Date: _____

Page No.: _____

TITLE

NumPy 2D Array – Branch Sales Analysis

OBJECTIVE

To use a 2D NumPy array for branch sales data and perform indexing, slicing and visualization to find weekly totals, branch averages, day-wise totals and the top branch.

AIM

To store sales data in a 2D array and compute branch/day-wise totals, averages and the highest-selling branch using NumPy.

THEORY

Sales matrix (3 branches $\times$ 7 days):

$$S = \begin{bmatrix} s_{11} & \cdots & s_{17} \\ s_{21} & \cdots & s_{27} \\ s_{31} & \cdots & s_{37} \end{bmatrix}$$

Indexing and slicing:

$$S[0], S[1, 3:], S[:, 0:3]$$

Weekly total per branch and grand total:

$$W_i = \sum_{j=1}^7 s_{ij} \quad (\text{axis} = 1) \quad T = \sum_{i,j} s_{ij}$$

Branch average and day-wise total:

$$\bar{B}_i = \frac{1}{7} \sum_{j=1}^7 s_{ij} \quad D_j = \sum_{i=1}^3 s_{ij} \quad (\text{axis} = 0)$$

Top branch:

$$k = \arg \max_i W_i$$

PROCEDURE

1. Store the sales data of 3 branches over 7 days in a 2D NumPy array and display it.
2. Display the 1st branch sales, Mon–Wed sales and 2nd-branch sales from Thursday.
3. Find weekly total per branch, grand total, branch-wise average and day-wise total.
4. Find the branch with the highest sales, display all results and plot the daily-sales bar and pie charts.

DATASET DESCRIPTION

Dataset Name: Branch sales (entered in program)

Source: 3 branches $\times$ 7 days (Mon–Sun), synthetic figures

Number of Samples: 21 entries Number of Features: 7 days

Target / Class: Not applicable (aggregation operations)

Description: B1=[1200, 1350, 1100, 1450, 1600, 2000, 1800]; B2=[900, 1100, 950, 1200, 1300, 1700, 1500]; B3=[1500, 1600, 1400, 1750, 1900, 2200, 2100].

PROGRAM

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

days = ["Mon", "Tue", "Wed", "Thu", "Fri", "Sat", "Sun"]

sales = np.array([
    [1200, 1350, 1100, 1450, 1600, 2000, 1800],
    [900, 1100, 950, 1200, 1300, 1700, 1500],
    [1500, 1600, 1400, 1750, 1900, 2200, 2100],
])

print("Complete sales data (branches x days):")
print(sales)

print("1st branch sales =", sales[0])

print("Mon-Wed sales:")
print(sales[:, 0:3])

print("2nd branch from Thursday =", sales[1, 3:])

print("Weekly total per branch =", sales.sum(axis=1))
print("Grand total =", sales.sum())

print("Branch-wise average =", np.round(sales.mean(axis=1), 2))

print("Day-wise total =", sales.sum(axis=0))

top = int(sales.sum(axis=1).argmax())
print("Highest sales: branch", top + 1)
x = np.arange(len(days))
w = 0.25
plt.figure(figsize=(7, 4.5))
plt.bar(x - w, sales[0], width=w, label="Branch 1")
plt.bar(x, sales[1], width=w, label="Branch 2")
plt.bar(x + w, sales[2], width=w, label="Branch 3")
plt.xticks(x, days)
plt.xlabel("Day")
plt.ylabel("Sales")
plt.title("Daily Sales per Branch")
plt.legend()
plt.tight_layout()
plt.savefig("exp4_sales.png", dpi=150)
plt.figure(figsize=(5, 4))
plt.pie(sales.sum(axis=1), labels=["Branch 1", "Branch 2", "Branch 3"], autopct="%1.1f%%")
plt.title("Branch Share of Total Sales")
plt.tight_layout()
plt.savefig("exp4_pie.png", dpi=150)
```

OUTPUT

Complete sales data (branches x days):

```
[[1200 1350 1100 1450 1600 2000 1800]
 [ 900 1100  950 1200 1300 1700 1500]
 [1500 1600 1400 1750 1900 2200 2100]]
```

1st branch sales = [1200 1350 1100 1450 1600 2000 1800]

Mon-Wed sales:

```
[[1200 1350 1100]
 [ 900 1100  950]
 [1500 1600 1400]]
```

2nd branch from Thursday = [1200 1300 1700 1500]

Weekly total per branch = [10500 8650 12450]

Grand total = 31600

Branch-wise average = [1500. 1235.71 1778.57]

Day-wise total = [3600 4050 3450 4400 4800 5900 5400]

Highest sales: branch 3

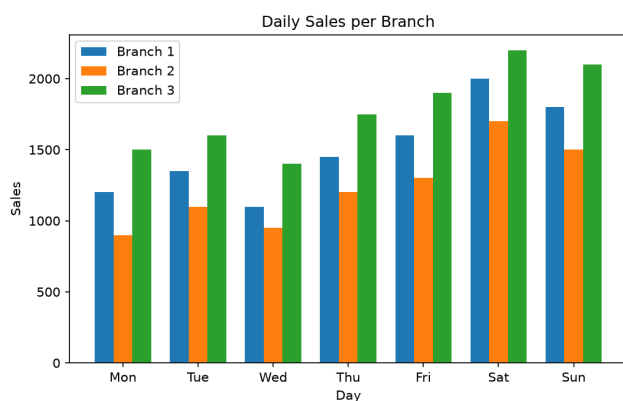


Fig. 1: Daily sales per branch.

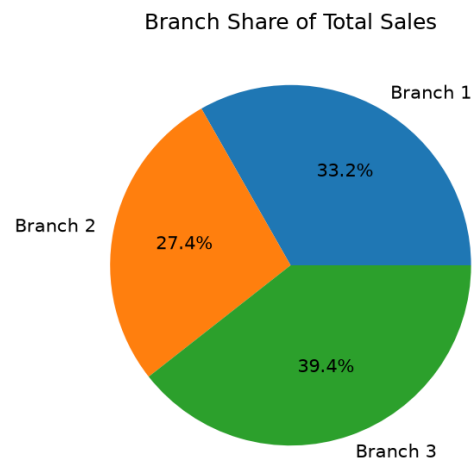


Fig. 2: Branch share of total sales.

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **NumPy data visualization** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 3A

Date: _____

Page No.: _____

TITLE

Simple Linear Regression – Study Hours vs Exam Score

OBJECTIVE

To implement simple linear regression on study-hours data by the regression equation and the standard (normal) equation, predict the score for 9 hours, and plot the regression line.

AIM

To fit $\hat{Y} = mX + b$ on hours-vs-score data with least squares and the normal equation, predict the 9-hour score, and evaluate with MSE and R^2 .

THEORY

Regression equation:

$$\hat{Y} = mX + b$$

Least-squares estimates:

$$m = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} \quad b = \bar{Y} - m\bar{X}$$

Prediction and goodness of fit:

$$\hat{Y}(9) = m \cdot 9 + b \quad \text{MSE} = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2 \quad R^2 = 1 - \frac{\sum (Y_i - \hat{Y}_i)^2}{\sum (Y_i - \bar{Y})^2}$$

PROCEDURE

1. Store hours (X) and scores (Y) of 8 students in NumPy arrays.
2. Compute m, b by least squares and by the normal equation.
3. Predict the score for 9 study hours with both equations.
4. Compute MSE and R^2 ; plot data points with the regression line.

DATASET DESCRIPTION

Dataset Name: Study hours vs exam score (given table)

Source: 8 students; X = hours (1–8), Y = score out of 100

Number of Samples: 8 Number of Features: 1 (hours)

Target / Class: Exam score (0–100)

Description: Y = [45, 50, 54, 60, 65, 70, 74, 80]; predict at X = 9.

PROGRAM

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

X = np.array([1, 2, 3, 4, 5, 6, 7, 8], dtype=float)
Y = np.array([45, 50, 54, 60, 65, 70, 74, 80], dtype=float)

x_bar, y_bar = X.mean(), Y.mean()
m = ((X - x_bar) * (Y - y_bar)).sum() / ((X - x_bar) ** 2).sum()
b = y_bar - m * x_bar
print(f"Least squares: m = {m:.4f}  b = {b:.4f}")

Xb = np.c_[np.ones(len(X)), X]
t = np.linalg.inv(Xb.T @ Xb) @ Xb.T @ Y
print(f"Normal equation: b = {t[0]:.4f}  m = {t[1]:.4f}")

print(f"LS prediction for 9 hrs = {m * 9 + b:.2f}")
print(f"NE prediction for 9 hrs = {t[1] * 9 + t[0]:.2f}")

for nm, (mm, bb) in [("LS", (m, b)), ("NE", (t[1], t[0]))]:
    p = mm * X + bb
    mse = float(((Y - p) ** 2).mean())
    r2 = float(1 - ((Y - p) ** 2).sum() / ((Y - y_bar) ** 2).sum())
    print(f"{nm}: MSE={mse:.4f} R2={r2:.4f}")

plt.figure(figsize=(7, 4.5))
plt.scatter(X, Y, s=40, label="Data points")
xs = np.linspace(0, 10, 200)
plt.plot(xs, m * xs + b, linewidth=2, label="Regression line")
plt.plot(9, m * 9 + b, marker="*", markersize=14, label="Prediction (9 hrs)")
plt.xlabel("Hours study (X)")
plt.ylabel("Exam score (Y)")
plt.title("Simple Linear Regression: Study Hours vs Exam Score")
plt.legend()
plt.tight_layout()
plt.savefig("exp5_fit.png", dpi=150)
```

OUTPUT

Least squares: $m = 4.9762$ $b = 39.8571$
Normal equation: $b = 39.8571$ $m = 4.9762$
LS prediction for 9 hrs = 84.64
NE prediction for 9 hrs = 84.64
LS: $MSE=0.1845$ $R^2=0.9986$
NE: $MSE=0.1845$ $R^2=0.9986$

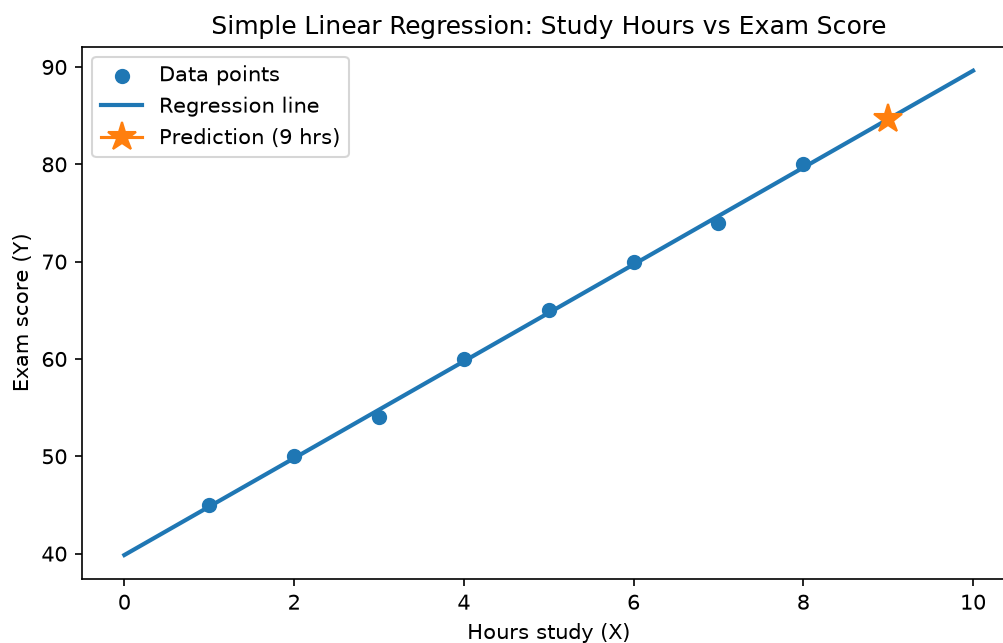


Fig. 1: Data points, regression line and 9-hour prediction.

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **least squares regression** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 3B

Date: _____

Page No.: _____

TITLE

Multiple Linear Regression – House Price from Area and Age

OBJECTIVE

To predict house selling price from area and age using multiple linear regression with the normal equation, and evaluate it with MSE and R-squared with a 3D plot.

AIM

To fit $\hat{Y} = b_0 + b_1X_1 + b_2X_2$ on the 6-house data, predict a new house price, and assess the fit.

THEORY

Regression plane (two variables):

$$\hat{Y} = b_0 + b_1X_1 + b_2X_2$$

Normal equation with bias column $X = [\mathbf{1} \ X_1 \ X_2]$:

$$\theta = (X^T X)^{-1} X^T y, \quad \theta = [b_0, b_1, b_2]^T$$

Prediction and goodness of fit:

$$\hat{Y}_{new} = b_0 + b_1X_{1,new} + b_2X_{2,new} \quad \text{MSE} = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2 \quad R^2 = 1 - \frac{\sum (Y_i - \hat{Y}_i)^2}{\sum (Y_i - \bar{Y})^2}$$

PROCEDURE

1. Store Area (X_1), Age (X_2) and Price (Y) of 6 houses in NumPy arrays.
2. Build X with a bias column and solve $\theta = (X^T X)^{-1} X^T y$.
3. Predict the price of a new house (Area 2.8, Age 4).
4. Compute MSE and R^2 ; plot houses with the fitted plane in 3D.

DATASET DESCRIPTION

Dataset Name: House price vs area and age (given table)

Source: 6 houses; X_1 = area, X_2 = age (years), Y = price (lakhs)

Number of Samples: 6 Number of Features: 2

Target / Class: Selling price in lakhs

Description: Area=[1.0–3.5]; Age=[10–2]; Price=[20–43].

PROGRAM

```
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt

X1 = np.array([1.0, 1.5, 2.0, 2.5, 3.0, 3.5])
X2 = np.array([10, 8, 6, 5, 3, 2], dtype=float)
Y = np.array([20, 24, 29, 32, 38, 43], dtype=float)

Xb = np.c_[np.ones(len(X1)), X1, X2]
t = np.linalg.inv(Xb.T @ Xb) @ Xb.T @ Y
b0, b1, b2 = t
print(f"Plane: Y = {b0:.4f} + {b1:.4f}*Area + {b2:.4f}*Age")

p = Xb @ t
mse = float(((Y - p) ** 2).mean())
r2 = float(1 - ((Y - p) ** 2).sum() / ((Y - Y.mean()) ** 2).sum())
print(f"MLR: MSE={mse:.4f} R2={r2:.4f} RMSE={np.sqrt(mse):.4f}")

new = b0 + b1 * 2.8 + b2 * 4
print(f"Predicted price (Area 2.8, Age 4) = {new:.2f} lakhs")

fig = plt.figure(figsize=(7, 5))
ax = fig.add_subplot(111, projection="3d")
ax.scatter(X1, X2, Y, s=40, label="Houses")
a = np.linspace(0.8, 3.7, 20)
g = np.linspace(1, 11, 20)
A, G = np.meshgrid(a, g)
ax.plot_surface(A, G, b0 + b1 * A + b2 * G, alpha=0.4)
ax.scatter([2.8], [4], [new], s=80, marker="*",
           label="Prediction (2.8, 4)")
ax.set_xlabel("Area X1")
ax.set_ylabel("Age X2")
ax.set_zlabel("Price Y (lakhs)")
ax.set_title("Multiple Linear Regression: Price vs Area and Age")
ax.legend()
plt.tight_layout()
plt.savefig("exp6_plane.png", dpi=150)
```

OUTPUT

Plane: $Y = 10.4286 + 9.1429 \cdot \text{Area} + 0.0000 \cdot \text{Age}$

MLR: MSE=0.3810 R2=0.9938 RMSE=0.6172

Predicted price (Area 2.8, Age 4) = 36.03 lakhs

Multiple Linear Regression: Price vs Area and Age

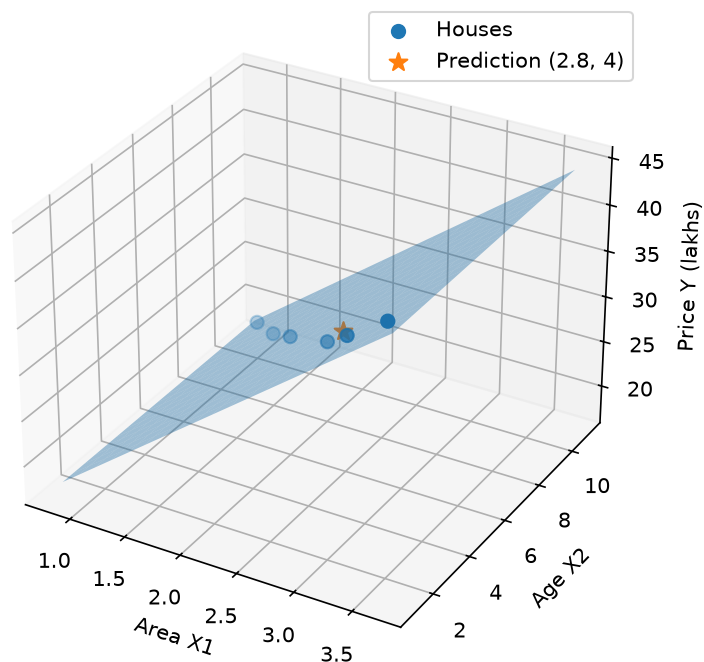


Fig. 1: Houses, fitted plane and predicted house (star).

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **multiple linear regression** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 4A

Date: _____

Page No.: _____

TITLE

Simple Linear Regression using Least Squares – TotalRooms

OBJECTIVE

To implement least squares regression on housing.csv (X=TotalRooms, Y=MedHouseVal) and evaluate it with MSE, R-squared and the regression-line plot.

AIM

To fit $\hat{Y} = mX + b$ by least squares on TotalRooms, predict test values, and assess the fit with MSE and R^2 .

THEORY

Regression equation:

$$\hat{Y} = mX + b$$

Least-squares estimates:

$$m = \frac{\sum_{i=1}^n (X_i - \bar{X})(Y_i - \bar{Y})}{\sum_{i=1}^n (X_i - \bar{X})^2} \quad b = \bar{Y} - m\bar{X}$$

Goodness of fit:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

PROCEDURE

- B) Load the California Housing dataset (housing.csv).
- C) Identify the variables: independent $X = \text{TotalRooms}$ (Total rooms); dependent $Y = \text{MedHouseVal}$ (Median House Value).
- D) Inspect the dataset for missing, duplicate, or invalid values.
- E) Calculate the slope m and the intercept b using the least square equation.
- F) Construct the regression equation $\hat{Y} = mX + b$.
- G) Use the regression equation to predict the target values for the test dataset.
- H) Calculate the Mean Squared Error (MSE) and the coefficient of determination R^2 to measure the goodness of fit.
- I) Plot the actual data points and regression line.

DATASET DESCRIPTION

Dataset Name: California Housing (housing.csv)

Source: Public census CSV file; prepared with NumPy only

Number of Samples: 20640 usable (0 missing) Number of Features: 1 (TotalRooms)

Target / Class: MedHouseVal – median house value in units of \$100,000

Description: Train 16512 / test 4128 (80/20, seed 42).

PROGRAM

```

import csv
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
rooms, prices, miss = [], [], 0
with open("housing.csv", newline="") as f:
    for r in csv.DictReader(f):
        try:
            rooms.append(float(r["total_rooms"]))
            prices.append(float(r["median_house_value"]))
        except (ValueError, KeyError):
            miss += 1
X = np.array(rooms)
y = np.array(prices) / 1e5
print("Usable:", len(X), " Missing:", miss)
rng = np.random.default_rng(42)
order = np.arange(len(X))
rng.shuffle(order)
n_train = int(0.8 * len(X))
X_train, X_test = X[order[:n_train]], X[order[n_train:]]
y_train, y_test = y[order[:n_train]], y[order[n_train:]]
print("Train:", len(X_train), " Test:", len(X_test))
x_bar = X_train.mean()
y_bar = y_train.mean()
m = ((X_train - x_bar) * (y_train - y_bar)).sum()
m = m / ((X_train - x_bar) ** 2).sum()
b = y_bar - m * x_bar
print(f"LS slope m = {m:.6f}")
print(f"LS intercept b = {b:.4f}")
y_pred = m * X_test + b
mse = ((y_test - y_pred) ** 2).mean()
r2 = 1 - ((y_test - y_pred) ** 2).sum() / ((y_test - y_test.mean()) ** 2).sum()
print(f"LS: MSE={mse:.4f} R2={r2:.4f} RMSE={np.sqrt(mse):.4f}")
plt.figure(figsize=(7, 4.5))
plt.scatter(X_test, y_test, s=6, alpha=0.25, label="Test data")
xs = np.linspace(0, float(np.percentile(X, 99.5)), 200)
plt.plot(xs, m * xs + b, linewidth=2, label="Least Squares line")
plt.xlabel("TotalRooms (X)")
plt.ylabel("MedHouseVal ($100k) (Y)")
plt.title("Simple Linear Regression (Least Squares): TotalRooms vs Price")
plt.legend()
plt.tight_layout()
plt.savefig("exp4a_fit.png", dpi=150)

```

OUTPUT

Usable: 20640 Missing: 0
Train: 16512 Test: 4128
LS slope $m = 0.000072$
LS intercept $b = 1.8840$
LS: MSE=1.3236 R2=0.0167 RMSE=1.1505

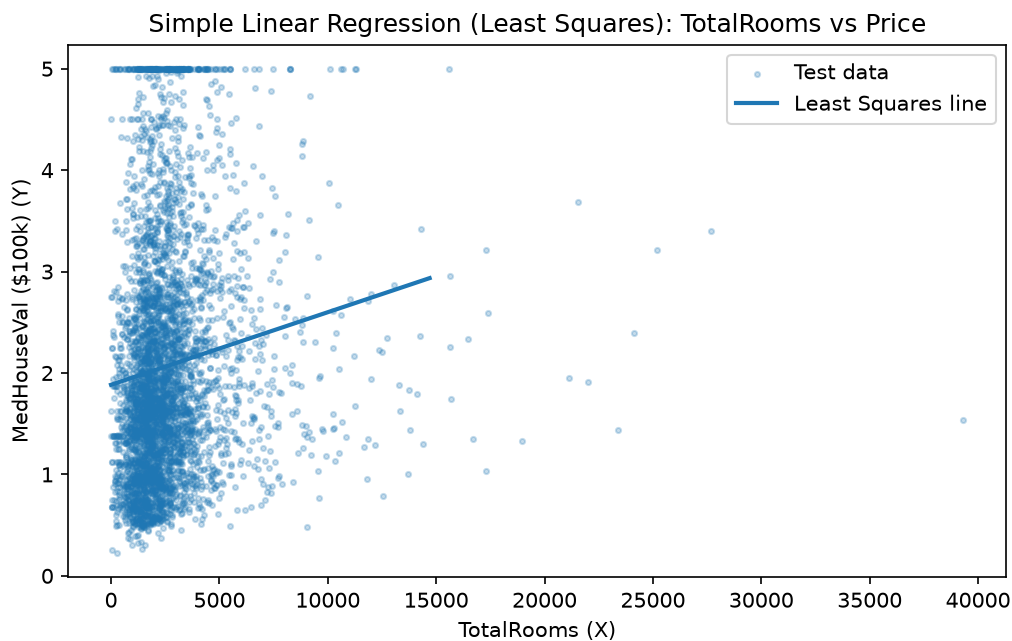


Fig. 1: Test data with the least-squares regression line.

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **least squares regression** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 4B

Date: _____

Page No.: _____

TITLE

Simple Linear Regression using Gradient Descent – TotalRooms

OBJECTIVE

To implement gradient descent regression on housing.csv (X=TotalRooms, Y=MedHouseVal) with $\alpha = 0.01$ and 100 epochs, and evaluate it with MSE, R-squared and the cost curve.

AIM

To fit $\hat{Y} = mX + b$ by gradient descent from $m = c = 0$, predict test values, and assess with MSE and R^2 .

THEORY

Linear hypothesis:

$$\hat{Y} = mX + b$$

Mean Squared Error cost:

$$J(m, c) = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$$

Gradients:

$$\frac{\partial J}{\partial m} = -\frac{2}{n} \sum_{i=1}^n X_i (Y_i - \hat{Y}_i) \quad \frac{\partial J}{\partial b} = -\frac{2}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)$$

Gradient descent updates:

$$m := m - \alpha \frac{\partial J}{\partial m} \quad b := b - \alpha \frac{\partial J}{\partial b}$$

PROCEDURE

1. Load the California Housing dataset and select TotalRooms as X and MedHouseVal as Y.
2. Inspect and clean the data by checking missing and invalid values.
3. Split the dataset into training and testing sets.
4. Standardize the input feature using the mean and standard deviation of the training data.
5. Initialize slope $m = 0$ and intercept $c = 0$.
6. Choose learning rate $\alpha = 0.01$ and epochs = 100.
7. Calculate the predicted values.
8. Calculate the Mean Squared Error cost $J(m, c) = \frac{1}{n} \sum (Y_i - \hat{Y}_i)^2$.
9. Calculate the gradient w.r.t. slope $\frac{\partial J}{\partial m} = -\frac{2}{n} \sum X_i (Y_i - \hat{Y}_i)$.
10. Calculate the gradient w.r.t. intercept $\frac{\partial J}{\partial b} = -\frac{2}{n} \sum (Y_i - \hat{Y}_i)$.
11. Update slope $m = m - \alpha \frac{\partial J}{\partial m}$ and intercept $b = b - \alpha \frac{\partial J}{\partial b}$.
12. Repeat steps 8–11 for 100 epochs or until the cost converges.
13. Plot cost versus epochs to verify convergence.
14. Predict test values with final m, b ; calculate MSE and R^2 .

DATASET DESCRIPTION

Dataset Name: California Housing (housing.csv)

Source: Public census CSV file; prepared with NumPy only; feature standardized

Number of Samples: 20640 usable (0 missing) Number of Features: 1 (TotalRooms)

Target / Class: MedHouseVal — median house value in units of \$100,000

Description: Train 16512 / test 4128 (80/20, seed 42).

PROGRAM

```

import csv
import numpy as np
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
rooms, prices, miss = [], [], 0
with open("housing.csv", newline="") as f:
    for r in csv.DictReader(f):
        try:
            rooms.append(float(r["total_rooms"]))
            prices.append(float(r["median_house_value"]))
        except (ValueError, KeyError):
            miss += 1
X = np.array(rooms)
y = np.array(prices) / 1e5
print("Usable:", len(X), " Missing:", miss)
rng = np.random.default_rng(42)
order = np.arange(len(X))
rng.shuffle(order)
n_train = int(0.8 * len(X))
X_train, X_test = X[order[:n_train]], X[order[n_train:]]
y_train, y_test = y[order[:n_train]], y[order[n_train:]]
print("Train:", len(X_train), " Test:", len(X_test))
mu = X_train.mean()
sg = X_train.std()
Xs_train = (X_train - mu) / sg
m, b = 0.0, 0.0
n = len(Xs_train)
costs = []
for epoch in range(100):
    y_hat = m * Xs_train + b
    err = y_train - y_hat
    J = (err ** 2).mean()
    costs.append(float(J))
    dm = (-2 / n) * (Xs_train * err).sum()
    db = (-2 / n) * err.sum()
    m = m - 0.01 * dm
    b = b - 0.01 * db
m_gd = m / sg
b_gd = b - m * mu / sg
print(f"GD slope m = {m_gd:.6f}")
print(f"GD intercept b = {b_gd:.4f}")
print(f"GD cost first={costs[0]:.4f} last={costs[-1]:.4f}")
y_pred = m_gd * X_test + b_gd
mse = ((y_test - y_pred) ** 2).mean()
r2 = 1 - ((y_test - y_pred) ** 2).sum() / ((y_test - y_test.mean()) ** 2).sum()
print(f"GD: MSE={mse:.4f} R2={r2:.4f} RMSE={np.sqrt(mse):.4f}")
plt.figure(figsize=(6, 3.8))
plt.plot(range(1, 101), costs)
plt.xlabel("Epoch")
plt.ylabel("Cost J (MSE)")
plt.title("Gradient Descent: Cost vs Epochs")
plt.tight_layout()
plt.savefig("exp4b_cost.png", dpi=150)

```

OUTPUT

Usable: 20640 Missing: 0
Train: 16512 Test: 4128
GD slope $m = 0.000062$
GD intercept $b = 1.6341$
GD cost first=5.6208 last=1.3827
GD: MSE=1.3885 R2=-0.0315 RMSE=1.1784

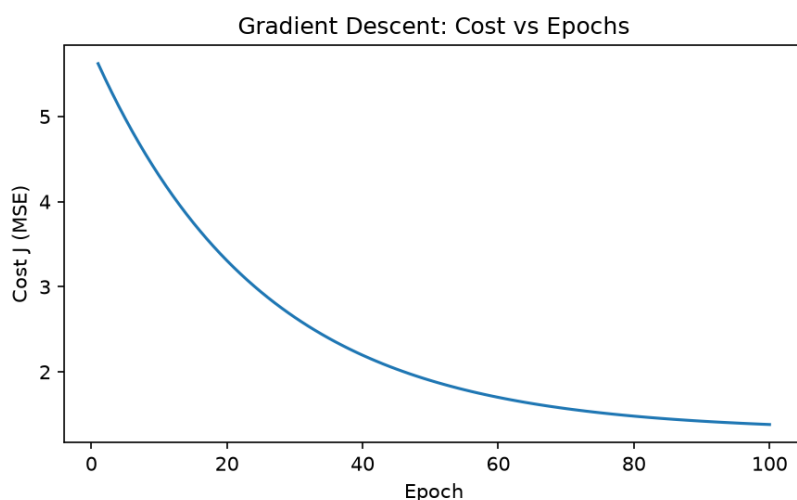


Fig. 2: Gradient-descent cost vs epochs ($\alpha = 0.01$, 100 epochs).

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **gradient descent regression** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____

EXPERIMENT NO. 4C

Date: _____

Page No.: _____

TITLE

Simple Linear Regression with Scikit-learn – AveRooms

OBJECTIVE

To implement simple linear regression with Scikit-learn on the California Housing dataset (X=AveRooms, Y=MedHouseVal) and evaluate it with MSE, R-squared and the regression-line plot.

AIM

To fit the sklearn LinearRegression model on AveRooms, predict test values, and assess with MSE and R^2 .

THEORY

Linear hypothesis:

$$\hat{Y} = mX + b$$

Scikit-learn fits m, b by least squares (normal equation via SVD). Goodness of fit:

$$\text{MSE} = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2 \quad R^2 = 1 - \frac{\sum_{i=1}^n (Y_i - \hat{Y}_i)^2}{\sum_{i=1}^n (Y_i - \bar{Y})^2}$$

PROCEDURE

1. Import NumPy and Pandas for data handling; fetch_california_housing, train_test_split, LinearRegression, mean_squared_error, r2_score and Matplotlib.
2. Load with fetch_california_housing(); build the DataFrame; assign feature names; separate the target.
3. Explore: first records, shape, features and missing values.
4. Take X=AveRooms, Y=MedHouseVal; scatter AveRooms vs MedHouseVal.
5. Split 80/20 (random_state=42); fit LinearRegression; read slope m and intercept b .
6. Predict test values; report MSE and R^2 ; plot test points with the regression line, labels, title and legend.

DATASET DESCRIPTION

Dataset Name: California Housing via Scikit-learn

Source: fetch_california_housing (1990 census)

Number of Samples: 20640 Number of Features: 8 (X=AveRooms)

Target / Class: MedHouseVal – median house value in units of \$100,000

Description: Train 16512 / test 4128 (80/20, random_state=42).

PROGRAM

```
import numpy as np
import pandas as pd
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import mean_squared_error, r2_score
housing = fetch_california_housing()
df = pd.DataFrame(housing.data, columns=housing.feature_names)
print("Shape:", df.shape)
print("Missing:", int(df.isnull().sum().sum()))
X = df[["AveRooms"]]
y = housing.target
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)
print("Train:", len(X_train), " Test:", len(X_test))
model = LinearRegression()
model.fit(X_train, y_train)
print(f"m = {model.coef_[0]:.6f} b = {model.intercept_: .4f}")
p = model.predict(X_test)
print(f"MSE={mean_squared_error(y_test, p):.4f} "
      f"R2={r2_score(y_test, p):.4f} RMSE={np.sqrt(mean_squared_error(y_test, p)):.4f}")
plt.figure(figsize=(7, 4.5))
plt.scatter(X_test, y_test, s=6, alpha=0.25, label="Test data")
xs = np.linspace(0, 15, 200)
plt.plot(xs, model.coef_[0] * xs + model.intercept_,
         linewidth=2, label="Sklern line")
plt.xlim(0, 15)
plt.xlabel("AveRooms")
plt.ylabel("MedHouseVal ($100k)")
plt.title("Simple Linear Regression with Scikit-learn")
plt.legend()
plt.tight_layout()
plt.savefig("exp4c_fit.png", dpi=150)
```

OUTPUT

```
Shape: (20640, 8)
Missing: 0
Train: 16512 Test: 4128
m = 0.076756 b = 1.6548
MSE=1.2923 R2=0.0138 RMSE=1.1368
```

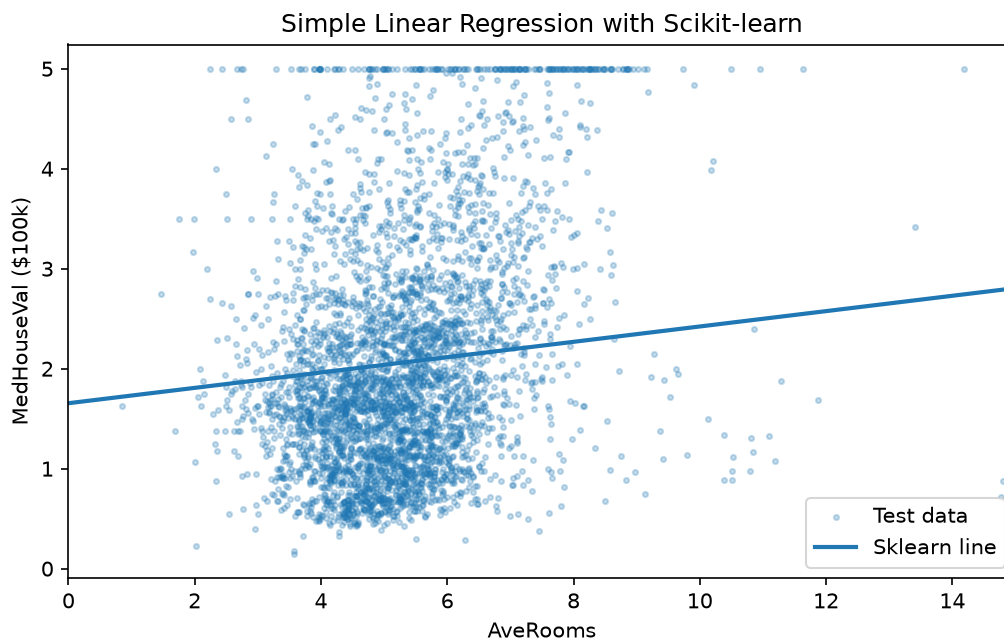


Fig. 1: Test data with the Scikit-learn regression line.

RESULT

The above program was implemented and executed successfully using Python, and the corresponding output was obtained and verified. The results of the **Scikit-learn LinearRegression model** were analyzed and found to be satisfactory, thereby achieving the desired objective of the experiment.

Faculty Signature: _____

Date: _____